

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 10

Part 0

Testing Essentials for Microservices

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Part 0.1 — Introduction to Software Testing

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

By the end of this chapter, you will be able to:

- Understand what software testing is and why it is an essential part of software development.
 - Explain the difference between testing and debugging.
 - Distinguish between verification and validation.
 - Understand the cost of software defects and why early testing saves time and money.
 - Explain why automated testing has become a fundamental practice in modern software engineering.
 - Understand the role of testing in professional software development before learning JUnit, Mockito, Spring Boot Testing, and TestContainers.
-

1. What Is Software Testing?

Software testing is the process of checking whether a software application behaves as expected.

Whenever we develop software, we have expectations:

- A calculator should add numbers correctly.
- A login page should reject incorrect passwords.
- An online store should not allow customers to purchase products that are out of stock.
- A payment service should reject invalid credit cards.

Testing verifies that the software satisfies these expectations.

Definition

Software Testing is the process of verifying and validating that software behaves correctly according to its requirements and continues to work correctly after changes are made.

Notice that the definition contains two important words:

- **Verifying**
- **Validating**

We will discuss both shortly.

Why Is Testing Necessary?

Imagine that you have developed an E-Commerce platform consisting of:

- Product Service
- Order Service
- Inventory Service
- Payment Service
- Notification Service

The application contains:

- Hundreds of Java classes
- Dozens of REST APIs
- Thousands of lines of code

One day, another developer changes only **three lines of code** inside the Inventory Service.

Question:

How can we be sure that this small change did not break something else?

Without testing, we cannot.

Software is constantly changing, and every change introduces the possibility of new defects.

Testing provides confidence that existing functionality still works after modifications.

What Happens Without Testing?

Consider this simple example.

A developer modifies the discount calculation.

Original code:

```
public double calculatePrice(double price) {  
    return price;  
}
```

Later, another developer adds a 10% discount.

```
public double calculatePrice(double price) {  
    return price * 0.9;  
}
```

Everything appears correct.

Several weeks later, another developer accidentally changes it again.

```
public double calculatePrice(double price) {  
    return price * 0.09;  
}
```

The application still compiles successfully.

There are no syntax errors.

However, customers now receive a **91% discount** instead of **10%**.

Without testing, this bug may remain unnoticed until customers begin placing orders.

With automated tests, the mistake would be detected immediately.

Testing Is Not Only About Finding Bugs

Many beginners believe that testing exists only to discover bugs.

Professional software engineers see testing differently.

Testing helps us:

- Verify business requirements.
- Detect programming errors.

- Prevent regression bugs.
- Improve software quality.
- Refactor code safely.
- Deploy software with confidence.
- Document expected system behavior.

Testing is a **quality assurance activity**, not simply a bug hunting activity.

Testing as a Safety Net

Think of automated tests as a safety net used by trapeze artists.

Without the safety net:

- Every change feels risky.
- Developers become afraid to modify code.
- Small changes may introduce unexpected bugs.

With a strong safety net:

- Developers work confidently.
- Features can be added safely.
- Existing functionality remains protected.

A professional software project should never rely only on manual verification.

2. Testing vs Debugging

Many beginners confuse these two terms.

Although they are related, they are completely different activities.

Software Testing

Testing answers the question:

Does the software work correctly?

Testing executes the application and compares the actual behavior with the expected behavior.

Example:

Expected Result:

$5 + 3 = 8$

Actual Result:

$5 + 3 = 7$

The test fails.

Testing has discovered a defect.

Debugging

Debugging answers a different question:

Why did the software fail?

After a failed test, developers investigate the code.

They may:

- Read the source code.
- Use breakpoints.
- Inspect variables.
- Trace program execution.
- Fix the defect.

Testing finds the problem.

Debugging fixes the problem.

Example

Suppose the following test fails.

```
assertEquals(8, calculator.add(5,3));
```

The actual result is:

7

Testing reports:

Expected: 8

Actual: 7

Now the developer begins debugging.

After investigation, they discover:

```
return a + b - 1;
```

The bug is corrected.

The test is executed again.

This time the test passes.

Testing and debugging work together but perform different roles.

Testing vs Debugging

Testing

Detects defects

Usually automated

Reports failures

Answers "Does it work?"

Debugging

Finds the cause of defects

Usually performed by developers

Fixes failures

Answers "Why doesn't it work?"

3. Verification vs Validation

These two terms are frequently used in software engineering.

Although similar, they represent different concepts.

Verification

Verification asks:

Are we building the product correctly?

Verification checks whether the implementation follows the design and technical specifications.

Examples:

- Does the API return the correct HTTP status?
- Does the database contain the expected data?
- Does the algorithm produce the expected result?

Verification focuses on correctness.

Validation

Validation asks:

Are we building the correct product?

Validation ensures that the software actually satisfies user needs.

Example:

Suppose a customer requests:

"Allow users to reset forgotten passwords."

The development team implements:

"Change password after login."

The implementation may be technically correct.

Verification succeeds.

However, the customer cannot reset a forgotten password.

Validation fails because the wrong feature was implemented.

Verification vs Validation

Verification

Are we building the product correctly?

Technical correctness

Focuses on implementation

Mostly performed by developers

Both are essential for successful software development.

Validation

Are we building the correct product?

Business correctness

Focuses on customer requirements

Often involves users or stakeholders

4. The Cost of Software Defects

One of the most important principles in software engineering is:

The later a defect is discovered, the more expensive it becomes to fix.

Consider the following timeline.

Requirements



Design



Development



Testing



Production

Suppose a developer introduces a defect during development.

Case 1 — Found Immediately

The developer notices the problem within a few minutes.

Time required:

- Five minutes

Cost:

- Almost zero.
-

Case 2 — Found During Testing

A tester discovers the bug several days later.

Now the developer must:

- Investigate the issue.
- Reproduce the problem.
- Modify the code.
- Run all tests again.

The cost becomes much higher.

Case 3 — Found in Production

The application has already been deployed.

Customers begin reporting problems.

Now the team must:

- Investigate production logs.

- Roll back the deployment.
- Restore data.
- Prepare an emergency fix.
- Redeploy the application.

The cost becomes extremely high.

In some cases:

- Money is lost.
- Customer trust decreases.
- Company reputation is damaged.

Why Early Testing Matters

Finding defects early provides several advantages:

- Lower development cost.
- Faster debugging.
- Easier maintenance.
- More reliable software.
- Greater customer satisfaction.

Professional software teams therefore test continuously throughout development rather than waiting until the end of the project.

5. Why Automated Testing Matters

Imagine an application containing:

- 50 REST APIs
- 20 Microservices
- Thousands of business rules

A new feature is added.

Question:

Should a developer manually test every feature before every deployment?

The answer is clearly **No**.

Manual testing would be too slow.

Instead, automated tests execute in minutes.

Developer writes code



Run Automated Tests



All Tests Pass



Build Application



Deploy with Confidence

This process is repeated every time the application changes.

Automated testing makes modern Continuous Integration (CI) and Continuous Deployment (CD) possible.

Without automated testing, DevOps would be impractical.

Real-World Example

Suppose your team deploys the E-Commerce platform five times every day.

Each deployment requires checking:

- Product APIs
- Order APIs
- Inventory APIs
- Payment APIs
- Security
- Database operations

If manual testing takes two hours per deployment:

5 Deployments × 2 Hours = 10 Hours Daily

Clearly impossible.

Automated tests can perform the same verification in a few minutes.

This is why modern software companies invest heavily in automated testing.

Key Takeaways

After completing this chapter, you should remember the following principles:

- Software testing verifies that software behaves as expected.
- Testing is not only about finding bugs—it builds confidence in software quality.
- Testing and debugging are different activities: testing discovers problems, debugging identifies and fixes their causes.
- Verification ensures that the software is built correctly, while validation ensures that the correct software is built.
- Defects become significantly more expensive to fix when they are discovered late in the development lifecycle.
- Automated testing enables frequent deployments, continuous integration, and reliable software delivery.
- Professional software development relies on testing as an integral part of the development process, not as a final step before release.

What's Next?

Now that you understand **why testing is essential**, the next chapter will explain **how testing fits into the modern software development lifecycle**, including Continuous Integration (CI), Continuous Testing, Shift-Left Testing, and the role of automated tests in professional DevOps pipelines.

Part 0.2 — Testing in Modern Software Development

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

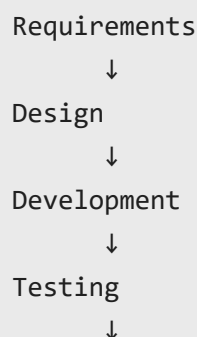
By the end of this chapter, you will be able to:

- Understand where testing fits into the Software Development Life Cycle (SDLC).
 - Explain why modern software teams test continuously instead of waiting until development is complete.
 - Understand Continuous Testing, Continuous Integration (CI), and Continuous Delivery (CD).
 - Explain the concept of Shift-Left Testing.
 - Understand how automated tests support DevOps and Microservices.
 - Understand why testing is considered an engineering activity rather than a final quality assurance step.
-

1. Testing in the Software Development Life Cycle (SDLC)

Many beginners imagine that testing is the final phase of software development.

For example:



Deployment

This was common many years ago.

Today, this approach is no longer suitable for modern software systems.

Applications change continuously, sometimes several times every day.

If testing only begins after development is complete, defects remain hidden for a long time, making them more expensive and more difficult to fix.

Instead, testing should be performed throughout the entire software development process.

Traditional Development vs Modern Development

Traditional Approach

Requirements



Design



Coding



Testing



Release

Characteristics:

- Testing starts late.
 - Bugs are discovered after most of the code is written.
 - Fixing defects is expensive.
 - Releases happen infrequently.
-

Modern Software Development

```
graph TD; A[Requirements] --> B[Development]; B --> C[Automated Tests]; C --> D[Build]; D --> E[Deploy]; E --> F[Repeat];
```

Requirements
↓
Development
↓
Automated Tests
↓
Build
↓
Deploy
↓
Repeat

Characteristics:

- Testing happens continuously.
- Developers receive immediate feedback.
- Small defects are fixed quickly.
- Software can be released frequently.

Why Continuous Testing?

Suppose your team develops an E-Commerce platform.

During one week:

- A new discount feature is added.
- Payment processing is updated.
- Inventory synchronization is improved.
- Product search is optimized.
- Security vulnerabilities are patched.

Question:

Should the team wait until Friday to test everything?

No.

By Friday:

- Hundreds of files may have changed.
- Many developers have modified the code.
- Finding the source of a bug becomes much harder.

Instead, every small change should be verified immediately.

This approach is called **Continuous Testing**.

2. Continuous Testing

Continuous Testing means:

Automatically testing software whenever changes are made.

Instead of waiting until the end of development, automated tests run continuously.

Typical workflow:

```
graph TD; A[Developer writes code] --> B[Save Changes]; B --> C[Run Unit Tests]; C --> D[Commit Code]; D --> E[Push to Git]; E --> F[CI Server Runs All Tests]; F --> G[Deploy];
```

Every code change is verified before reaching production.

This significantly reduces the risk of introducing new defects.

Benefits of Continuous Testing

Continuous Testing provides many advantages:

- Immediate feedback.
- Faster bug detection.
- Lower maintenance costs.
- Higher software quality.
- Faster software delivery.
- Greater developer confidence.

Continuous Testing is one of the foundations of DevOps.

3. Continuous Integration (CI)

Continuous Integration (CI) is the practice of frequently merging code changes into a shared repository.

Every time a developer pushes code:

- The application is compiled.
- Automated tests are executed.
- Build quality is verified.

If everything succeeds, the project remains healthy.

If a test fails, developers are notified immediately.

Continuous Integration Workflow



```
|  
├─ Compile Project  
├─ Run Unit Tests  
├─ Run Integration Tests  
├─ Generate Reports  
└─ Build Docker Image
```

If any step fails, the deployment process stops.

This prevents defective software from reaching production.

Why CI Is Important

Imagine a team of ten developers.

Every day:

- Everyone changes different parts of the application.
- New features are added.
- Existing bugs are fixed.

Without CI:

- Conflicts remain unnoticed.
- Bugs accumulate.
- Integration becomes difficult.

With CI:

Every code change is verified immediately.

Problems are discovered while they are still small.

4. Continuous Delivery (CD)

Continuous Delivery extends Continuous Integration.

After the application passes all automated tests:

```
Compile
  ↓
Run Tests
  ↓
Build Docker Image
  ↓
Deploy to Test Environment
  ↓
Ready for Production
```

The software is always in a deployable state.

Deployment to production may still require human approval.

Continuous Deployment

Some companies go one step further.

After all tests pass successfully:

```
Developer Push
  ↓
Compile
  ↓
Tests
  ↓
Build
  ↓
Deploy Automatically
```

No manual approval is required.

This approach is called **Continuous Deployment**.

Large technology companies often deploy software many times every day using this strategy.

CI vs CD

Continuous Integration

Merge code frequently

Run automated tests

Detect integration problems

Continuous Delivery

Always ready to release

Deploy to staging

Human approval

Continuous Deployment

Automatically released

Deploy to production

No human approval

5. Shift-Left Testing

One of the most important ideas in modern software engineering is **Shift-Left Testing**.

Traditionally:

```
Requirements
  ↓
Design
  ↓
Coding
  ↓
Testing
```

Testing occurs near the end.

Shift-Left moves testing earlier.

```
Requirements
  ↓
Design + Testing
  ↓
Coding + Testing
  ↓
Deployment
```

Developers test their code while they are writing it.

Problems are found much earlier.

Why Shift-Left Works

Suppose a developer introduces a bug.

Traditional Approach

The bug is discovered:

- Two weeks later.
- Hundreds of new changes have already been added.

Finding the cause becomes difficult.

Shift-Left Approach

The bug is discovered:

- Five minutes after writing the code.

The developer remembers exactly what changed.

Fixing the problem is quick and inexpensive.

6. Testing in DevOps

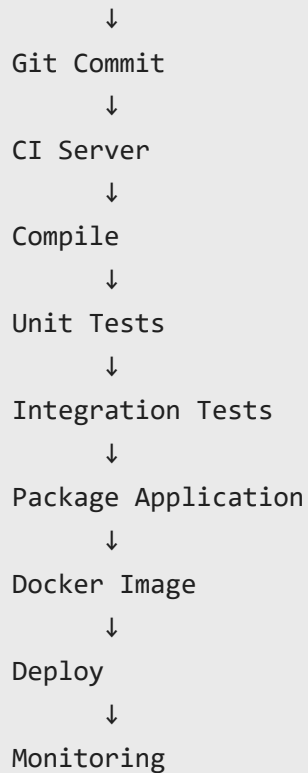
DevOps combines software development and IT operations.

One of its main goals is to deliver software quickly and reliably.

This is impossible without automated testing.

A typical DevOps pipeline looks like this:

Developer



Notice that testing happens before deployment.

If tests fail, deployment stops automatically.

Why Testing Is Critical in Microservices

Our E-Commerce platform contains multiple independent services:

Product Service
Order Service
Inventory Service
Payment Service
Notification Service

Each service evolves independently.

Different teams may deploy changes at different times.

This creates new challenges:

- APIs change.
- Databases change.
- Services communicate over the network.
- Failures may occur between services.

Without automated testing, these changes become extremely difficult to manage.

Testing allows each service to evolve safely while reducing the risk of breaking the entire platform.

7. Testing Is Everyone's Responsibility

Years ago, testing was often considered the responsibility of a dedicated QA (Quality Assurance) team.

Today, modern software teams follow a different philosophy:

Quality is everyone's responsibility.

Developers:

- Write unit tests.
- Write integration tests.
- Fix failed tests.
- Improve test coverage.

QA Engineers:

- Design broader test strategies.
- Perform exploratory testing.
- Validate business requirements.
- Automate end-to-end scenarios.

DevOps Engineers:

- Integrate automated tests into CI/CD pipelines.
- Monitor test execution.
- Ensure reliable deployments.

Everyone contributes to software quality.

Engineering Perspective

Professional software engineers do not ask:

"Can I finish writing the code?"

Instead, they ask:

"Can I prove that my code works correctly?"

A feature is not considered complete until it has been tested.

This mindset is one of the characteristics of professional software development.

Key Takeaways

After completing this chapter, you should remember the following:

- Testing is a continuous activity throughout software development.
 - Modern software teams test early and test often.
 - Continuous Testing provides immediate feedback after every code change.
 - Continuous Integration automatically builds and tests software whenever developers push new code.
 - Continuous Delivery ensures software is always ready for deployment.
 - Shift-Left Testing reduces the cost of fixing defects by detecting them early.
 - Automated testing is a critical part of DevOps and Microservices.
 - Software quality is the responsibility of the entire development team.
-

Preparing for the Next Chapter

Now that you understand **where testing fits into modern software development**, the next chapter will introduce the **different types of software testing**.

You will learn:

- Unit Testing
- Integration Testing
- System Testing
- End-to-End (E2E) Testing
- Acceptance Testing

These concepts provide the foundation for understanding the **Testing Pyramid**, which is the first major topic in **Session 10**.

Part 0.3 — Types of Software Testing

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

By the end of this chapter, you will be able to:

- Understand the major categories of software testing.
 - Distinguish between Unit, Integration, System, End-to-End, Acceptance, and Contract Testing.
 - Understand the purpose of each testing type.
 - Identify when each type should be used.
 - Explain why no single testing type is sufficient for professional software development.
 - Understand how these testing types fit into a Microservices architecture.
-

1. Why Are There Different Types of Testing?

Suppose you buy a new car.

Before it reaches the customer, many different tests are performed.

For example:

- The engine is tested.
- The brakes are tested.
- The steering system is tested.
- The complete vehicle is driven on the road.
- Safety tests are performed.

Each test has a different purpose.

Software is exactly the same.

We do **not** write one giant test for the entire application.

Instead, we write different kinds of tests, each answering a different question.

For example:

Question	Test Type
Does this method work correctly?	Unit Test
Can these components work together?	Integration Test
Does the complete application work?	System Test
Can a customer complete a purchase?	End-to-End Test
Does the software satisfy customer requirements?	Acceptance Test
Can two microservices communicate correctly?	Contract Test

Each testing type increases confidence from a different perspective.

2. Unit Testing

Unit Testing verifies the smallest testable piece of software.

A **unit** is usually:

- A method
- A class
- A small business component

Unit tests execute one class in complete isolation.

Example:

```
ProductService
```

```
↓
```

```
Mock ProductRepository
```

```
↓
```

Business Logic

The database is **not** involved.

REST APIs are **not** involved.

Other microservices are **not** involved.

Only the business logic is tested.

Example

Suppose ProductService contains:

```
public double calculateDiscount(double price) {  
    return price * 0.90;  
}
```

A Unit Test simply checks:

Input:

100

↓

Expected Output:

90

Nothing else is required.

Characteristics

- Very fast

- Easy to write
 - Easy to maintain
 - Executes in milliseconds
 - Does not require Spring Boot
 - Does not require a database
-

Advantages

- ✓ Fast execution
 - ✓ Excellent for business logic
 - ✓ Easy debugging
 - ✓ Immediate developer feedback
-

Limitations

A Unit Test cannot verify:

- Database configuration
 - REST Controllers
 - Spring Dependency Injection
 - HTTP Communication
 - PostgreSQL behavior
 - Kafka messaging
-

3. Integration Testing

Integration Testing verifies that multiple software components work correctly together.

Instead of testing one class, Integration Tests verify collaboration.

Example:

Controller
↓

Service
↓
Repository
↓
PostgreSQL

Now the application behaves much more like production.

Example

When a client sends:

POST /products

The following sequence occurs:

Controller

↓

Service

↓

Repository

↓

Database

An Integration Test verifies the entire flow.

Characteristics

- Slower than Unit Tests.

- Requires Spring Boot.
 - May require a database.
 - Tests communication between components.
-

Advantages

- ✓ High confidence
 - ✓ Detects configuration problems
 - ✓ Detects dependency injection issues
 - ✓ Verifies SQL queries
-

Limitations

- Slower
 - More resources required
 - More difficult to debug
-

Unit Test vs Integration Test

Unit Test	Integration Test
One class	Multiple components
No Spring Context	Spring Context
No database	Real or embedded database
Very fast	Medium speed
Low resource usage	Higher resource usage
Both are essential.	

Professional projects require both.

4. System Testing

System Testing verifies the complete application as a whole.

Everything is running:

- Controllers
- Services
- Database
- Security
- Configuration

The application is treated as a complete system.

Example:

Client

↓

REST API

↓

Spring Boot Application

↓

Database

The internal implementation is not important.

Only the external behavior matters.

Example

Suppose the Product Service exposes:

```
GET /products
```

A System Test verifies:

- HTTP Request
- Controller
- Service
- Repository
- Database
- JSON Response

Everything is tested together.

5. End-to-End (E2E) Testing

End-to-End Testing verifies an entire business workflow from the user's perspective.

Unlike System Testing, an End-to-End Test may involve multiple applications.

Example:

Customer purchases a product.

Customer

↓

Product Service

↓

Order Service

↓

Inventory Service

↓

Payment Service

↓

Notification Service

↓

Customer receives email

The entire business process is verified.

Characteristics

- Highest confidence
 - Very slow
 - Expensive to maintain
 - Covers real user scenarios
-

Example

A complete purchase process:

Login

↓

Browse Products

↓

Add to Cart

↓

Checkout

↓

Payment

↓

Inventory Updated

↓

Confirmation Email

If any step fails, the test fails.

6. Acceptance Testing

Acceptance Testing answers one important question:

Does the software satisfy the customer's requirements?

Unlike Unit Tests, Acceptance Tests focus on business expectations.

Example:

Requirement:

Customers should receive free shipping for orders over \$100.

Acceptance Test:

Order Total:

\$120

↓

Expected Shipping Cost:

\$0

The business rule is verified.

7. Contract Testing (Preview)

Contract Testing is especially important in Microservices.

Suppose:

Order Service calls:

Inventory Service

Order Service

↓

HTTP

↓

Inventory Service

The API contract specifies:

```
{  
  "productId": 10,  
  "available": true  
}
```

One day, Inventory Service changes the response:

```
{
```

```
"id": 10,  
"stock": true  
}
```

Both services compile successfully.

Both Unit Tests pass.

Production fails because the API contract has changed.

Contract Testing prevents this problem.

We will study Contract Testing in detail in **Session 11**.

8. Which Test Should We Write?

There is no single correct answer.

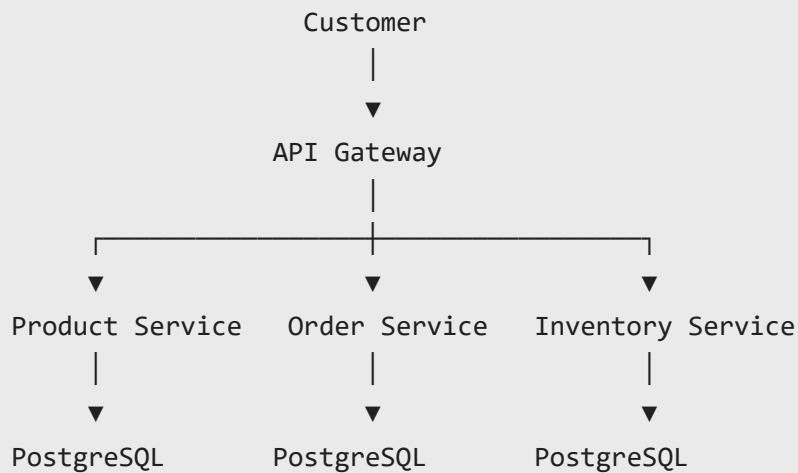
Different problems require different tests.

Problem	Best Test
Business calculation	Unit Test
Repository query	Integration Test
REST Controller	Integration Test
Complete Purchase Flow	End-to-End Test
Customer Requirement	Acceptance Test
API compatibility between services	Contract Test

Professional engineers choose the simplest test that can detect the problem.

9. How These Tests Fit in Our E-Commerce Platform

Let's revisit our project.



Different testing types focus on different parts of the platform:

Unit Testing

Tests one service in isolation.

Example:

ProductService

↓

Business Logic

Integration Testing

Tests collaboration inside one microservice.

Example:

Controller



Service



Repository



Database

Contract Testing

Tests communication between microservices.

Example:

Order Service



HTTP



Inventory Service

End-to-End Testing

Tests the entire platform.

Example:

Customer



API Gateway



Order Service



Inventory Service



Payment Service



Notification Service

Each testing type answers a different engineering question.

Engineering Decision

A common mistake made by beginners is trying to solve every problem with one type of test.

For example:

- Writing only Unit Tests.
- Writing only End-to-End Tests.
- Ignoring Integration Tests.

Professional software systems combine multiple testing types.

Each provides a different level of confidence.

Key Takeaways

After completing this chapter, you should remember:

- Software testing consists of multiple testing types, each with a different purpose.
- Unit Tests verify individual classes and business logic.
- Integration Tests verify collaboration between application components.
- System Tests verify the complete application.
- End-to-End Tests verify complete business workflows from the user's perspective.
- Acceptance Tests verify that business requirements are satisfied.
- Contract Tests verify communication between independent microservices.
- No single testing type is sufficient for professional software development.
- Choosing the correct testing type is an engineering decision based on what you want to verify.

What's Next?

Now that you understand the different **types of software testing**, the next chapter will explain **why testing becomes more challenging in a Microservices architecture**.

We will compare **Monolithic Applications** and **Microservices**, explore the new types of failures introduced by distributed systems, and understand why **Unit Tests alone are no longer enough**. This chapter provides the perfect bridge into **Session 10 (Testing Pyramid, JUnit 5, Mockito, Spring Test Slices, and TestContainers)** and **Session 11 (Contract Testing, WireMock, and Chaos Engineering)**.

Part 0.4 — Why Testing Changes in Microservices

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

By the end of this chapter, you will be able to:

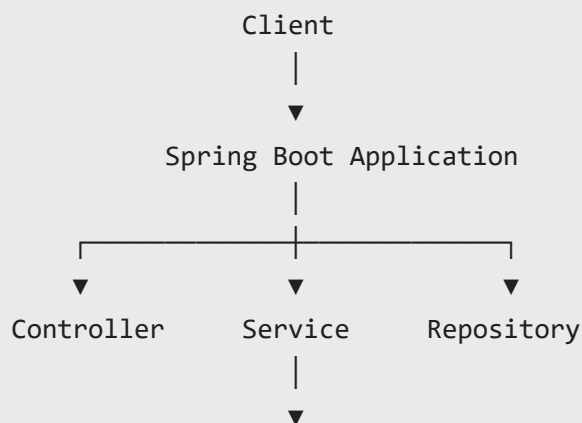
- Explain why testing a microservices architecture is more challenging than testing a monolithic application.
- Identify the new failure points introduced by distributed systems.
- Understand why Unit Tests alone are insufficient in microservices.
- Explain why Integration Testing, Contract Testing, and Chaos Engineering become essential.
- Understand the testing journey that will be covered in Sessions 10 and 11.

1. Testing a Monolithic Application

Before learning how to test microservices, let's understand how testing works in a traditional monolithic application.

A monolithic application contains all business logic inside a single application.

Example:



PostgreSQL

Everything runs inside one application.

Communication between components happens through **Java method calls**.

Example:

controller → service → repository

There is:

- No network communication.
- No remote servers.
- No HTTP calls between services.
- No service discovery.

Testing is relatively straightforward.

Typical Monolithic Test

Suppose a client sends:

POST /products

The request flows like this:

HTTP Request



Controller



Service



Repository



Database

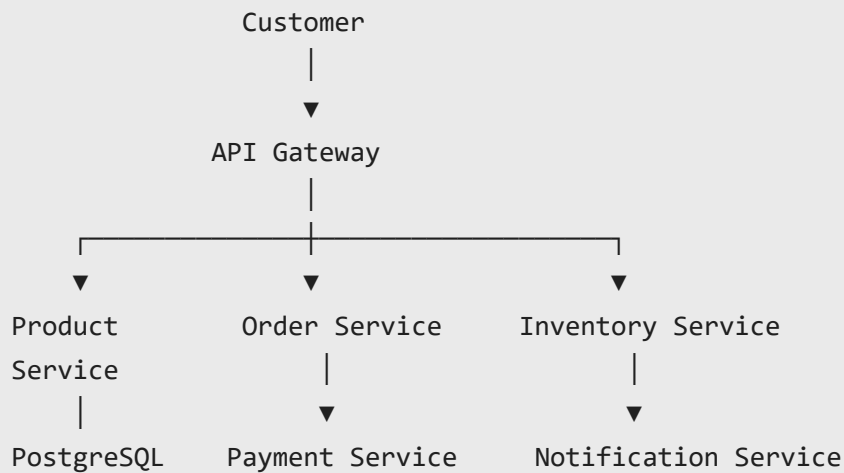
If something fails, the cause is usually inside the same application.

Unit and Integration Tests are often enough to detect most problems.

2. Testing a Microservices Application

Now consider our E-Commerce platform.

Instead of one application, we have multiple independent services.



Each service:

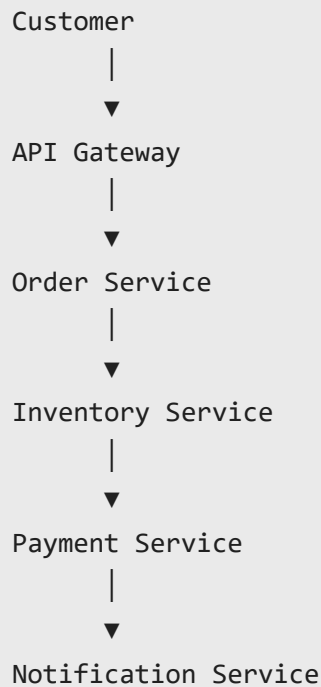
- Runs independently.
- Has its own database.
- Can be deployed separately.
- May be developed by different teams.

The architecture is much more flexible—but also much more complex to test.

A Simple Purchase Request

Suppose a customer places an order.

The request may travel through multiple services.



Even if each individual service works perfectly, the complete workflow may still fail.

Why?

Because failures can occur **between services**.

3. New Types of Failures in Microservices

In a monolithic application, most failures occur inside the application itself.

In a microservices architecture, failures may occur anywhere in the communication path.

Let's look at some common examples.

Network Failure

The Order Service sends an HTTP request.

```
graph TD; Order_Service[Order Service] -->
```



Inventory Service

What happens if the network connection is temporarily unavailable?

The request never reaches the Inventory Service.

Your business logic may be correct.

Your Unit Tests may pass.

Production still fails.

Service Unavailable

Suppose the Inventory Service is stopped for maintenance.

Order Service



Inventory Service

🔌 Offline

Should the Order Service crash?

Of course not.

It should fail gracefully.

This is exactly why we learned about:

- Circuit Breakers
- Retry
- Bulkhead

earlier in the course.

Slow Response

Sometimes the service is available but very slow.

Order Service

|



Inventory Service

Response Time

15 seconds

Should users wait forever?

No.

The application should detect timeouts and respond appropriately.

Incorrect API Response

Yesterday the Inventory Service returned:

```
{  
  "productId": 10,  
  "available": true  
}
```

Today another developer changes the API:

```
{  
  "id": 10,  
  "stock": true  
}
```

The Inventory Service works correctly.

The Order Service works correctly.

Both Unit Tests pass.

But the services can no longer communicate.

This problem cannot be detected by Unit Tests.

Database Differences

Consider the following scenario.

During development:

H2 Database

In production:

PostgreSQL

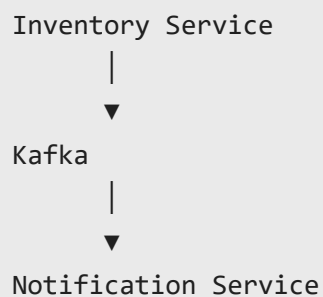
A Repository Test passes successfully using H2.

The same SQL query fails in PostgreSQL because of differences in SQL behavior, constraints, or data types.

This is exactly why Session 10 introduces **TestContainers**.

Message Broker Problems

Suppose the Inventory Service publishes an event.



Possible problems:

- Message never arrives.
- Wrong event format.
- Consumer cannot deserialize the message.
- Duplicate messages.
- Messages arrive out of order.

Again, Unit Tests cannot detect these problems.

4. Why Unit Tests Are Not Enough

Imagine every microservice has:

- 100% Unit Test coverage
- All Unit Tests pass

Can we safely deploy?

Many beginners would answer:

Yes.

Professional engineers answer:

Not necessarily.

Why?

Because Unit Tests only verify the code inside one service.

They do **not** verify:

- HTTP communication
- JSON serialization
- API compatibility
- Database behavior
- Spring configuration
- Network failures
- Service discovery
- Kafka messaging

Unit Tests provide confidence—but only for a small part of the system.

Example

Suppose the following services exist:

Order Service

Inventory Service

Payment Service

Each team writes excellent Unit Tests.

Order Service

✓ 100% Pass

Inventory Service

✓ 100% Pass

Payment Service

✓ 100% Pass

Everything looks perfect.

Now the Order Service sends:

```
{  
  "productId": 5  
}
```

The Inventory Service expects:

```
{  
  "id": 5  
}
```

The request fails.

All Unit Tests remain green.

Production fails.

Engineering Insight

This is one of the biggest lessons in Microservices.

A collection of perfectly tested services does not guarantee a perfectly working system.

The communication between services must also be tested.

5. Which Testing Types Solve These Problems?

Different problems require different tests.

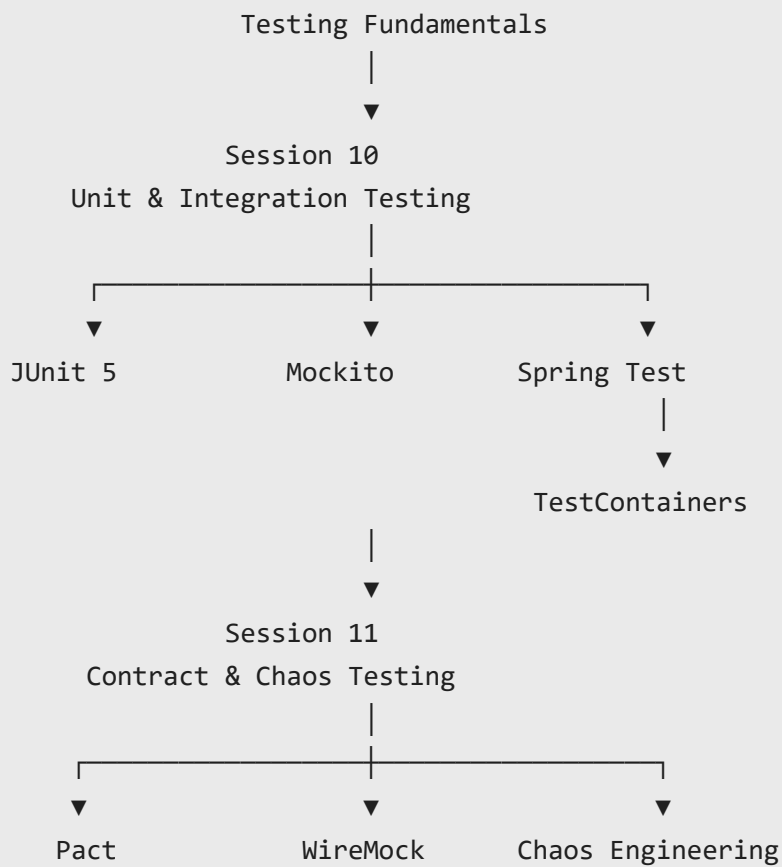
Problem	Best Testing Type
Business logic inside one class	Unit Test
Controller → Service → Repository	Integration Test
Repository with real PostgreSQL	TestContainers Integration Test
HTTP communication between services	Contract Test
External REST API simulation	WireMock
Service unavailable	Chaos Engineering
Complete customer purchase	End-to-End Test

Notice how no single testing type solves every problem.

Professional systems combine multiple testing strategies.

6. The Testing Journey in This Course

The remainder of this course builds testing knowledge step by step.



Each session addresses a different level of testing.

7. Engineering Decision

A professional software engineer does **not** ask:

"Which testing framework should I use?"

Instead, they ask:

"What problem am I trying to detect?"

The problem determines the testing approach.

Examples:

Problem

Solution

Problem	Solution
Verify a calculation	Unit Test
Verify Spring MVC validation	<code>@WebMvcTest</code>
Verify Repository queries	<code>@DataJpaTest</code>
Verify PostgreSQL behavior	TestContainers
Verify HTTP communication	Pact
Simulate an external REST service	WireMock
Verify resilience during failures	Chaos Engineering

This engineering mindset is far more valuable than memorizing framework annotations.

Key Takeaways

After completing this chapter, you should remember:

- Testing microservices is fundamentally different from testing monolithic applications.
- Distributed systems introduce new failure points such as network issues, service outages, API incompatibilities, database differences, and messaging problems.
- Unit Tests verify only the internal logic of a single service.
- A microservices platform requires multiple testing layers, including Integration Testing, Contract Testing, and Chaos Engineering.
- Different testing strategies solve different engineering problems.
- Choosing the correct testing approach is an engineering decision based on the type of risk you want to reduce.

Preparing for Session 10

You now have the necessary background to begin **Session 10 – Testing: Unit & Integration**.

In Session 10, we will answer an important question:

Why do some applications have 100% passing tests but still fail in production?

We will start with a real-world production story and then learn how the **Testing Pyramid** helps us decide **which type of test should be written for each problem**, before exploring **JUnit 5**, **Mockito**, **Spring Boot Test Slices**, and **TestContainers**.

Excellent. We have now built the foundation. The next chapter is where we teach students **how professional engineers think about testing**, before they learn any framework.

I consider this chapter one of the most valuable in the entire course because it teaches **decision making**, not annotations.

Part 0.5 — Testing Strategy & Engineering Principles

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

By the end of this chapter, you will be able to:

- Understand that testing is an engineering strategy, not just writing test cases.
 - Explain the relationship between confidence, speed, and cost.
 - Understand why no application should rely on only one type of test.
 - Learn the principles behind the Testing Pyramid.
 - Understand the characteristics of a good software test.
 - Apply engineering decision-making when choosing a testing approach.
-

1. Testing Is a Strategy, Not a Tool

Many beginners think testing means learning:

- JUnit
- Mockito
- Spring Boot Test
- TestContainers

These are **tools**, not the strategy.

Professional engineers think differently.

Before writing a test, they ask:

What am I trying to prove?

Only after answering that question do they choose a testing framework.

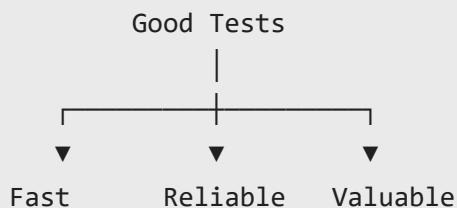
For example:

Question	Tool
Does this method calculate the correct discount?	JUnit
Does this service call the repository correctly?	Mockito
Does the REST controller validate input correctly?	<code>@WebMvcTest</code>
Does the SQL query work with PostgreSQL?	TestContainers
Can two microservices communicate correctly?	Pact
Does the application survive service failures?	Chaos Engineering

Notice that the **problem determines the tool**, not the other way around.

2. The Three Engineering Goals

Every software test attempts to balance three important goals.



A good test should be:

- Fast
- Reliable
- Valuable

If one of these is missing, the test loses much of its usefulness.

Fast

Fast tests execute quickly.

Example:

- Unit Test

Execution time:

10–50 milliseconds

Advantages:

- Developers run them frequently.
 - Immediate feedback.
 - Suitable for every code change.
-

Reliable

Reliable tests produce the same result every time.

Good tests never behave like this:

Monday

✓ Pass

Tuesday

✗ Fail

Wednesday

✓ Pass

These unstable tests are called **Flaky Tests**.

Flaky tests reduce trust in the testing process.

Professional teams work hard to eliminate flaky tests.

Valuable

A test should verify meaningful behavior.

Poor example:

```
assertNotNull(productService);
```

This provides almost no value.

Better example:

```
assertEquals(90, service.calculateDiscount(100));
```

This verifies an actual business rule.

3. Confidence vs Cost

Every software test provides confidence.

However, confidence has a cost.

Consider the following comparison.

Test Type	Execution Time	Cost	Confidence
Unit Test	Very Fast	Low	Medium
Integration Test	Medium	Medium	High
Contract Test	Medium	Medium	High
End-to-End Test	Slow	High	Very High

Notice:

Higher confidence usually requires:

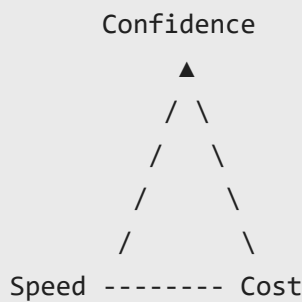
- More infrastructure
- More execution time

- More maintenance

This is why professional teams combine different testing types.

4. The Trade-Off Triangle

Testing always involves trade-offs.



Increasing one aspect often affects another.

For example:

Unit Test

- Very fast
 - Low cost
 - Moderate confidence
-

Integration Test

- Slower
 - Higher cost
 - Greater confidence
-

End-to-End Test

- Slowest
- Highest cost
- Highest confidence

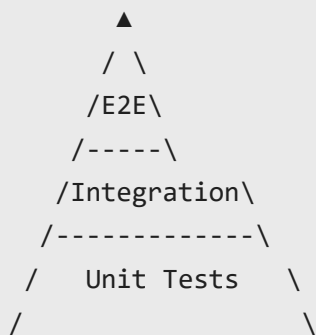
Professional engineers balance these trade-offs rather than maximizing only one.

5. The Testing Pyramid

One of the most influential ideas in software testing is the **Testing Pyramid**.

It suggests that applications should have:

- Many Unit Tests
- Fewer Integration Tests
- Very Few End-to-End Tests



Each level has a different purpose.

Unit Tests (Base of the Pyramid)

Characteristics:

- Fast
- Cheap
- Easy to maintain

Write many of these.

They protect business logic.

Integration Tests (Middle)

Characteristics:

- Moderate speed
- Higher confidence
- Verify collaboration between components

Write enough to verify important interactions.

End-to-End Tests (Top)

Characteristics:

- Slow
- Expensive
- Difficult to maintain

Write only for the most critical business workflows.

Example:

- Customer checkout
 - Payment processing
 - User registration
-

Why the Pyramid Matters

Suppose a project contains:

5 Unit Tests

100 End-to-End Tests

Every code change requires running many slow tests.

Development becomes slow.

Now consider:

500 Unit Tests

40 Integration Tests

10 End-to-End Tests

Most defects are detected quickly.

Only the most important workflows require expensive testing.

This is a much healthier testing strategy.

6. Characteristics of a Good Test

Professional software tests share common characteristics.

Simple

A test should verify one behavior.

Poor example:

- Tests ten unrelated scenarios.

Good example:

- Tests one business rule.

Independent

Each test should run independently.

Bad practice:

Test B depends on Test A.

If Test A fails, Test B also fails.

Good practice:

Each test prepares its own data.

Repeatable

Running the same test multiple times should always produce the same result.

Never depend on:

- Current date
- Current time
- Network availability
- Existing database records

unless that is the purpose of the test.

Readable

Good tests should read almost like documentation.

Example:

```
shouldCalculateDiscountForPremiumCustomer()
```

is much better than

```
test1()
```

Maintainable

Tests should be easy to update when business requirements change.

Avoid:

- Duplicate code
- Hard-coded values
- Large setup methods

7. Common Testing Mistakes

Beginners often make the following mistakes.

Mistake 1

Testing multiple behaviors in one test.

Bad:

Login

Payment

Search

Cart

Checkout

One Test

If it fails, which behavior is broken?

No one knows.

Mistake 2

Using a real database in every Unit Test.

Result:

Slow tests.

Instead:

Use mocks.

Mistake 3

Ignoring failed tests.

Some developers think:

"The application still works."

Professional teams never ignore failing tests.

A failing automated test should always be investigated.

Mistake 4

Testing framework code instead of business logic.

Example:

Testing whether Spring injects a bean correctly in every Unit Test.

Spring Framework already tests its own behavior.

Focus on your application logic.

Mistake 5

Trying to achieve 100% code coverage at any cost.

High coverage is useful.

Meaningful coverage is better.

A project with:

- 80% meaningful coverage

is often better than

- 100% meaningless coverage.

8. Engineering Decision Guide

One of the most valuable habits of a professional engineer is selecting the right testing approach.

Use this decision table whenever you need to write a test.

If you want to verify...	Recommended approach
One Java method	Unit Test + JUnit
Business logic with dependencies	Unit Test + Mockito
REST Controller behavior	<code>@WebMvcTest</code>
Repository queries	<code>@DataJpaTest</code>
Complete Spring Boot application	<code>@SpringBootTest</code>
PostgreSQL-specific behavior	TestContainers
Communication between services	Pact
External REST service	WireMock
Platform resilience	Chaos Engineering

This table summarizes the testing journey covered in Sessions 10 and 11.

Engineering Principles to Remember

Professional software engineers follow these principles:

Principle 1

A passing test increases confidence, but it does not prove the software is bug-free.

A test only verifies the scenarios it was designed to check.

Principle 2

The closer a test is to production, the more confidence it provides—but the slower and more expensive it becomes.

Balance confidence with execution time and maintenance cost.

Principle 3

Use the simplest test that can detect the problem.

Don't write a slow End-to-End test if a Unit Test can verify the same behavior.

Principle 4

Test behavior, not implementation.

Verify what the application does, not how the code is written internally.

For example, test that a discount is calculated correctly—not that a specific private method was called.

Principle 5

Tests are production code.

Tests should follow the same quality standards as application code:

- Clear names
- Clean structure
- Readable code
- Proper refactoring
- Good documentation

Poor-quality tests become a maintenance burden.

Key Takeaways

After completing this chapter, you should remember:

- Testing is an engineering strategy, not just a set of tools.
 - Every test balances speed, confidence, and cost.
 - The Testing Pyramid recommends many Unit Tests, fewer Integration Tests, and only a small number of End-to-End Tests.
 - Good tests are simple, independent, repeatable, readable, and maintainable.
 - Professional engineers choose the testing approach based on the problem they want to detect.
 - Writing high-quality tests is just as important as writing high-quality production code.
-

Preparing for the Next Chapter

You now understand **why we test, where testing fits into modern software development, the different types of testing, why testing changes in microservices, and the engineering principles behind an effective testing strategy.**

In the next chapter, we will connect these concepts directly to this course by introducing the **Testing Roadmap**. This roadmap shows how the topics in **Session 10 (Unit & Integration Testing)** and **Session 11 (Contract & Chaos Testing)** fit into the complete testing lifecycle of our E-Commerce Microservices Platform.

Part 0.6 — Testing Roadmap for This Course

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

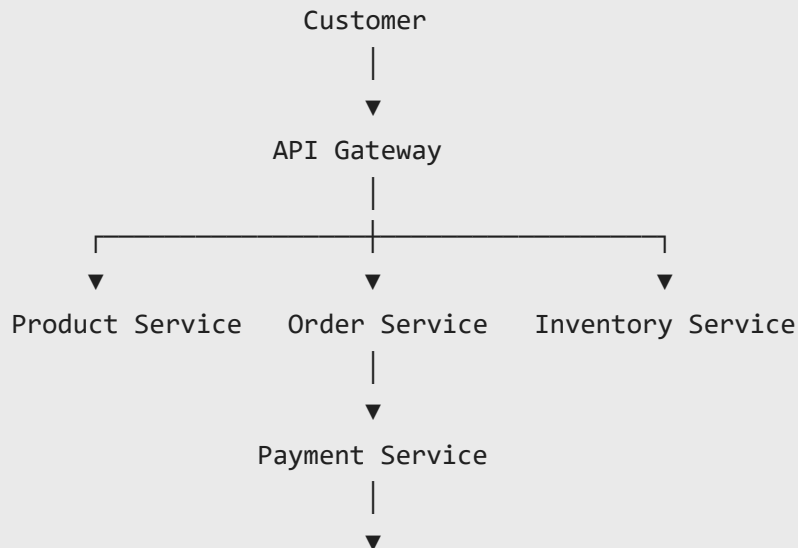
By the end of this chapter, you will be able to:

- Understand how testing will be implemented throughout this course.
 - Identify the purpose of each testing technology introduced in Sessions 10 and 11.
 - Understand how the different testing techniques complement each other.
 - Select the appropriate testing tool based on the engineering problem you want to solve.
-

1. The Big Picture

Throughout this course, we have built a complete E-Commerce Microservices Platform.

Our platform contains several independent services:



Notification Service

Each service contains:

- REST Controllers
- Business Logic
- Database Access
- Configuration
- External Communication

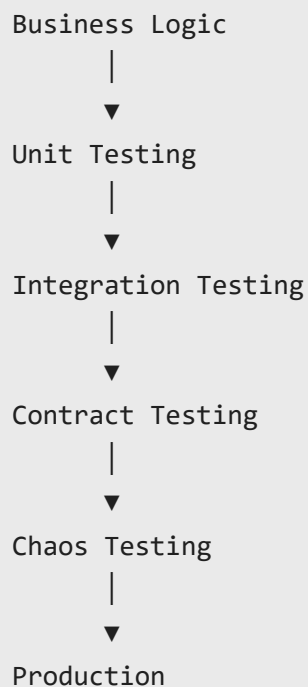
Testing every part of this platform requires different testing techniques.

No single testing framework can verify everything.

2. Our Testing Journey

During Sessions 10 and 11, we will gradually increase the confidence in our application.

Think of testing as building layers of protection.



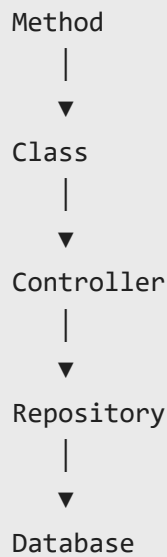
Each layer protects us from different kinds of problems.

3. Session 10 — Unit & Integration Testing

Session 10 focuses on verifying the correctness of a single microservice.

We begin by testing individual Java classes.

Later, we gradually test larger parts of the application.



The goal is to answer questions such as:

- Does this business rule work?
- Does the controller validate user input?
- Does Spring inject dependencies correctly?
- Does the repository execute the correct SQL?
- Does PostgreSQL behave exactly as expected?

Session 10 Technologies

During Session 10 we will learn:

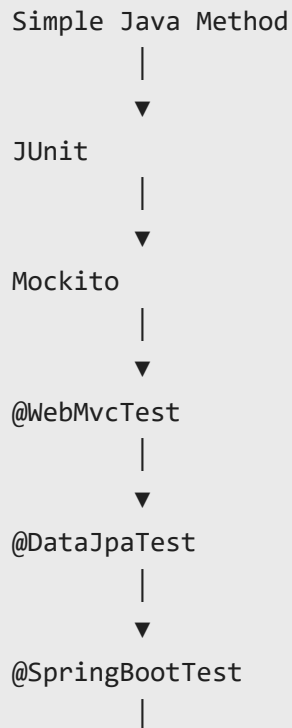
Technology	Purpose
------------	---------

Technology	Purpose
JUnit 5	Write and execute unit tests
Mockito	Replace dependencies with mock objects
Spring Boot Test	Load Spring context for testing
MockMvc	Test REST Controllers without running a web server
Test Slices	Load only the required Spring components
TestContainers	Run tests against a real PostgreSQL database

Notice the progression.

We start with the smallest unit of code and gradually move closer to a real production environment.

Session 10 Testing Progression



▼
TestContainers

Each step increases confidence.

Each step also increases complexity and execution time.

4. Session 11 — Contract & Chaos Testing

Session 11 expands our focus.

Instead of testing one service, we begin testing communication between services.

Order Service
|
▼
Inventory Service

Questions now become:

- Do both services agree on the same API?
- What happens if another service is unavailable?
- Can our application survive unexpected failures?

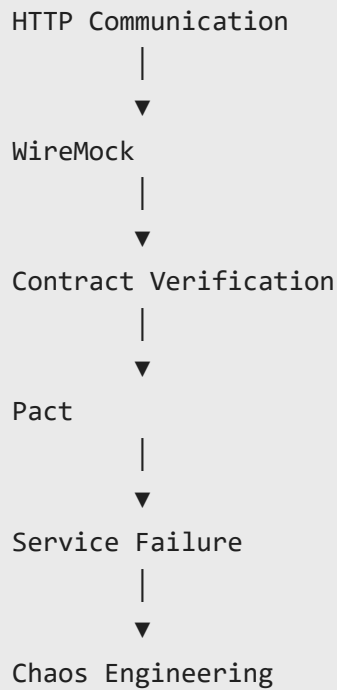
Session 11 Technologies

Technology	Purpose
Pact	Verify API contracts between microservices
WireMock	Simulate external HTTP services
Chaos Engineering	Verify system resilience during failures

Notice that these technologies do **not** replace Unit Tests or Integration Tests.

Instead, they verify problems that earlier testing layers cannot detect.

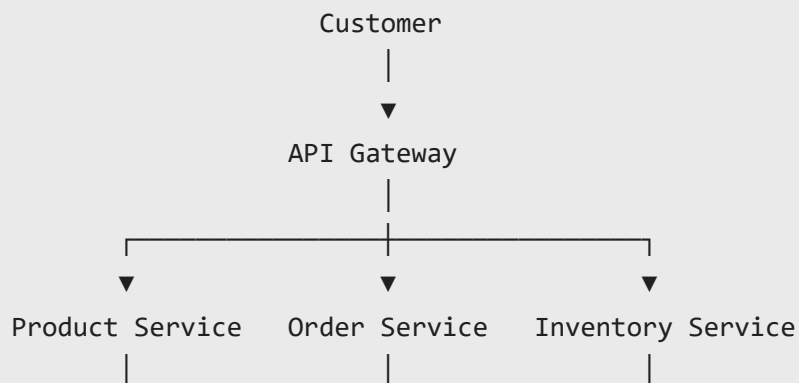
Session 11 Testing Progression



These tests prepare the application for real production environments.

5. How Everything Fits Together

The following diagram summarizes the complete testing strategy used in this course.



▼ PostgreSQL ▼ PostgreSQL ▼ PostgreSQL

Unit Test

|
|— Business Logic

Integration Test

|
|— Controller
|— Service
|— Repository
|— Database

Contract Test

|
|— Order Service ↔ Inventory Service
|— Order Service ↔ Payment Service

Chaos Test

|
|— Stop Inventory Service
|— Delay Payment Service
|— Network Failure
|— Observe System Behaviour

Each testing level focuses on a different engineering concern.

6. Testing Decision Matrix

As professional engineers, we should choose the testing technique based on the problem we are trying to solve.

Engineering Question	Recommended Tool
Does one Java method work correctly?	JUnit
Do I need to isolate dependencies?	Mockito
Does my REST Controller behave correctly?	<code>@WebMvcTest</code>
Does my Repository execute the correct query?	<code>@DataJpaTest</code>
Does the complete Spring application work?	<code>@SpringBootTest</code>
Does the application behave correctly with PostgreSQL?	TestContainers
Can two microservices communicate correctly?	Pact
How do I simulate another REST service?	WireMock
How does the system behave during failures?	Chaos Engineering

This table will become your quick reference throughout the remaining sessions.

7. Skills You Will Gain

By the end of Sessions 10 and 11, you will be able to:

- ✓ Write professional Unit Tests using JUnit 5.
- ✓ Isolate dependencies using Mockito.
- ✓ Test Spring MVC Controllers with MockMvc.
- ✓ Test Repository classes using `@DataJpaTest`.
- ✓ Run Integration Tests with a real PostgreSQL database using TestContainers.
- ✓ Verify API compatibility between microservices using Pact.

- ✓ Simulate external services using WireMock.
- ✓ Evaluate the resilience of a microservices platform using Chaos Engineering.

These are practical skills widely used in modern Spring Boot and cloud-native projects.

8. Engineering Mindset

Before moving to Session 10, remember these important principles:

Principle 1

Every bug should be caught by the cheapest test that can detect it.

If a Unit Test can detect the problem, there is no need to write an expensive End-to-End test.

Principle 2

Not everything should be tested in the same way.

Different problems require different testing strategies.

Principle 3

A testing framework is just a tool.

Your engineering decision should always come before choosing the framework.

Principle 4

Testing is an investment.

Writing tests requires time today, but it saves much more time when maintaining and evolving the application.

Summary of Part 0

Congratulations! You have completed the **Testing Fundamentals for Microservices**.

You now understand:

- Why software testing is essential.
- How testing fits into the modern software development lifecycle.
- The different types of software testing.
- Why testing becomes more challenging in a microservices architecture.
- The engineering principles behind an effective testing strategy.
- How Sessions 10 and 11 fit into the complete testing roadmap.

You are now ready to begin **Session 10 – Testing: Unit & Integration**, where you will apply these concepts using **JUnit 5**, **Mockito**, **Spring Boot Test Slices**, and **TestContainers**.

Before We Start Session 10...

There is one final practical prerequisite that will make the hands-on sessions much easier:

Part 0.7 — Getting Started with Java & Spring Boot Testing

In this chapter, we will build a complete testing workflow from scratch:

- Adding the required testing dependencies
- Understanding the Maven test structure
- Creating your first JUnit test
- Learning the Arrange–Act–Assert (AAA) pattern
- Using basic JUnit assertions
- Creating your first Mockito mock
- Running tests from IntelliJ IDEA and Maven
- Reading test results and debugging failures

This ensures that every student—regardless of their previous experience—starts Session 10 with the same practical foundation. I think this final chapter will make the transition into your detailed Session 10 materials much smoother.

Part 0.7 — Getting Started with Java & Spring Boot Testing

Course: Spring Boot & Spring Cloud — Professional Edition

Module: Testing Fundamentals for Microservices

Learning Objectives

By the end of this chapter, you will be able to:

- Understand the structure of a Java test project.
 - Add the required testing libraries to a Spring Boot project.
 - Create and run your first JUnit test.
 - Apply the Arrange–Act–Assert (AAA) testing pattern.
 - Use common JUnit assertions.
 - Create simple Mockito mocks.
 - Understand the difference between real objects and mocked objects.
 - Execute tests from IntelliJ IDEA and Maven.
 - Read and understand test results.
 - Prepare yourself for the advanced testing techniques introduced in Session 10.
-

1. Before We Write Our First Test

Writing tests follows the same general process regardless of the framework.

Every test begins with one question:

What behavior am I trying to verify?

For example:

- Does the `add()` method return the correct sum?
- Does the service calculate the correct discount?
- Does the repository save the entity?
- Does the controller reject invalid input?

Only after identifying the behavior do we write the test.

The Testing Workflow

Professional developers follow a simple workflow.

Write Production Code



Create Test Class



Prepare Test Data



Call the Method



Verify the Result



Run the Test



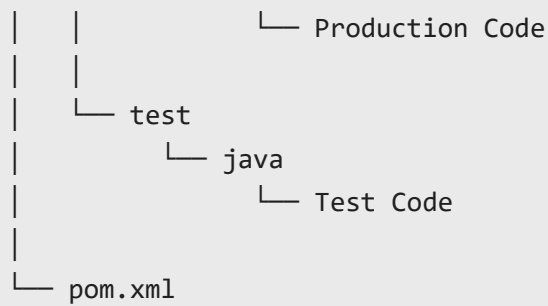
Fix Problems (if any)

Notice that testing is simply another part of software development—not a separate activity.

2. Project Structure

In a Maven-based Spring Boot project, the source code and test code are stored in different folders.

```
project
|
├─ src
|   └─ main
|       └─ java
```



Production Code

All application code is placed in:

```
src/main/java
```

Examples:

- Controllers
- Services
- Repositories
- Entities

Test Code

All test classes are placed in:

```
src/test/java
```

A common beginner mistake is placing test classes inside `src/main/java`.

Always keep test code separate from production code.

3. Adding Testing Dependencies

Spring Boot makes testing easy by providing a single starter dependency.

Inside `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

This single dependency automatically includes many testing libraries.

What's Included?

Library	Purpose
JUnit 5	Writing and running tests
Mockito	Creating mock objects
Spring Test	Spring testing support
MockMvc	Testing REST Controllers
AssertJ	Fluent assertions
Hamcrest	Matcher library

Notice that we only add **one dependency**, but Spring Boot manages several libraries for us.

4. Creating Your First JUnit Test

Suppose we have a simple class.

```
public class Calculator {

    public int add(int a, int b){
```

```
        return a + b;
    }

}
```

We want to verify that the `add()` method works correctly.

Create a new class named:

CalculatorTest

Inside `src/test/java`.

Our First Test

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {

    @Test
    void shouldAddTwoNumbers() {

        Calculator calculator = new Calculator();

        int result = calculator.add(5,3);

        assertEquals(8,result);

    }

}
```

Understanding Every Line

@Test

```
@Test
```

This annotation tells JUnit:

"This method is a test."

Without this annotation, JUnit ignores the method.

Create the Object

```
Calculator calculator = new Calculator();
```

We create an instance of the class we want to test.

Call the Method

```
int result = calculator.add(5,3);
```

We execute the production code.

Verify the Result

```
assertEquals(8,result);
```

Expected value:

8

Actual value:

result

If they match:

PASS

Otherwise:

FAIL

5. The AAA Pattern

Nearly every professional unit test follows the **AAA Pattern**.

AAA stands for:

- **Arrange**
- **Act**
- **Assert**

This simple structure makes tests easy to read and maintain.

Arrange

Prepare everything the test needs.

```
Calculator calculator = new Calculator();
```

Act

Execute the method being tested.

```
int result = calculator.add(5,3);
```

Assert

Verify that the result is correct.

```
assertEquals(8,result);
```

Complete Example

```
@Test
void shouldAddTwoNumbers() {

    // Arrange
    Calculator calculator = new Calculator();

    // Act
    int result = calculator.add(5,3);

    // Assert
    assertEquals(8,result);

}
```

As you write more tests, following the AAA structure will make your code easier to understand.

6. Common JUnit Assertions

JUnit provides many assertion methods.

Here are the ones you will use most frequently.

Assertion	Purpose
<code>assertEquals()</code>	Verify two values are equal
<code>assertNotEquals()</code>	Verify values are different
<code>assertTrue()</code>	Verify a condition is true
<code>assertFalse()</code>	Verify a condition is false
<code>assertNull()</code>	Verify an object is null
<code>assertNotNull()</code>	Verify an object is not null
<code>assertThrows()</code>	Verify an exception is thrown

Example

```
assertTrue(product.isAvailable());

assertFalse(cart.isEmpty());

assertNotNull(order);

assertThrows(
    IllegalArgumentException.class,
    () -> service.createProduct(null)
);
```

Don't try to memorize every assertion now. The goal is to recognize the most commonly used ones before Session 10.

7. Running Tests

From IntelliJ IDEA

Open the test class.

Click the green ► icon next to:

- The test method, or
- The test class.

IntelliJ executes the selected tests and displays the results.

From Maven

Run:

```
mvn test
```

Maven compiles the project and executes all tests under `src/test/java`.

A successful run typically ends with output similar to:

```
Tests run: 5
```

```
Failures: 0
```

```
Errors: 0
```

```
BUILD SUCCESS
```

8. Reading Test Results

When a test fails, don't panic. Read the output carefully.

For example:

```
Expected: 8
```

Actual: 7

This tells you exactly what the test expected and what it actually received.

The stack trace shows where the failure occurred.

Learning to read test failures is an important development skill.

9. Why Do We Need Mockito?

Consider the following class.

```
ProductService
  |
  ▼
ProductRepository
  |
  ▼
Database
```

Suppose we only want to test the business logic inside `ProductService`.

Should every unit test connect to the database?

No.

Doing so would make tests slower and more difficult to maintain.

Instead, we replace the real repository with a **mock**.

Real Object vs Mock Object

Without Mockito

ProductService



The mock behaves like the repository but does not connect to a database.

This keeps the test fast and focused.

10. Your First Mockito Example

Suppose `ProductService` depends on `ProductRepository`.

```
ProductRepository repository = Mockito.mock(ProductRepository.class);
```

This creates a fake repository.

We can now define its behavior.

```
when(repository.existsBySku("P100"))  
    .thenReturn(true);
```

Now, whenever the service calls:

```
repository.existsBySku("P100");
```

the mock returns:

```
true
```

No database is involved.

Verifying Interactions

Mockito can also verify that a method was called.

```
verify(repository).save(product);
```

This does **not** check the database.

Instead, it verifies that `ProductService` attempted to save the product.

Session 10 will explore Mockito features such as `@Mock`, `@Spy`, `ArgumentCaptor`, and `InOrder` in much greater detail.

11. Common Beginner Mistakes

Avoid these common errors:

- Placing test classes in `src/main/java`.
- Forgetting the `@Test` annotation.
- Testing multiple unrelated behaviors in one test.
- Using a real database in unit tests.
- Ignoring failing tests.
- Writing tests with unclear names such as `test1()` or `sampleTest()`.

Prefer descriptive names such as:

```
shouldCalculateDiscountForPremiumCustomer()
```

```
shouldRejectDuplicateSku()
```

```
shouldReturnProductWhenIdExists()
```

A good test name explains the expected behavior.

12. Before Session 10

Before moving to Session 10, make sure you can:

- ✓ Add `spring-boot-starter-test` to a project.
- ✓ Create a test class under `src/test/java`.
- ✓ Write a simple JUnit test using `@Test`.
- ✓ Apply the Arrange–Act–Assert pattern.
- ✓ Use basic JUnit assertions.
- ✓ Run tests from IntelliJ IDEA.
- ✓ Run tests using `mvn test`.
- ✓ Read a test failure message.
- ✓ Create a simple Mockito mock.
- ✓ Stub a method using `when(...).thenReturn(...)`.
- ✓ Verify interactions using `verify(...)`.

Key Takeaways

After completing this chapter, you should remember:

- Test code belongs in `src/test/java`, separate from production code.
 - Spring Boot simplifies testing with the `spring-boot-starter-test` dependency.
 - A typical test follows the Arrange–Act–Assert (AAA) pattern.
 - JUnit is used to write and execute tests, while Mockito replaces real dependencies with mock objects.
 - Unit tests should focus on one behavior and avoid unnecessary external dependencies.
 - Clear test names and readable test structure make tests easier to understand and maintain.
 - Running and interpreting test results is an essential developer skill.
-

Preparing for Session 10

You now have all the background knowledge required to begin **Session 10 – Testing: Unit & Integration**.

From this point onward, we will assume you are comfortable with:

- Writing basic JUnit tests.
- Creating Mockito mocks.
- Running tests successfully.
- Reading test failures.

This allows us to focus on the advanced topics that professional Spring Boot developers use every day:

- The **Testing Pyramid** and testing strategy.
- Advanced JUnit 5 features such as `@ParameterizedTest`.
- Advanced Mockito features including `@Spy`, `ArgumentCaptor`, and `InOrder`.
- Spring Boot Test Slices (`@WebMvcTest`, `@DataJpaTest`, and `@SpringBootTest`).
- **TestContainers** for running integration tests against a real PostgreSQL database.

With this foundation, every student in the class should be able to follow the live coding and understand not only **how** to write tests, but also **why** a particular testing approach is the right engineering choice.